

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Thursday 10th September, 2026

Chapter 1

Experiments

This chapter reports the experimental study of the Clustering Agent Representation (CAR) method introduced in Chapter ???. While the previous chapter presented CAR and its evaluation metrics from a theoretical standpoint, this chapter focuses on their concrete realization: the software used to run the experiments, the environments and training setups considered, and a detailed analysis of the results obtained.

This chapter is organized as follows:

- Section 1.1 describes the software infrastructure implementing the CAR pipeline.
- Section 1.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section 1.3 presents and analyses the results obtained by applying CAR and its metrics.
- Section 1.4 summarizes the findings and discusses their implications.

1.1 Software Infrastructure

From our point of view, deep learning is an experimental science: the value of a method cannot be established by its formulation alone, but only through experiments that are reliable, run at scale, and can be reproduced. In this setting, the quality of an implementation matters as much as the algorithm it runs. A method that cannot be executed reproducibly on realistic problems is not a usable contribution. We therefore give the implementation of CAR the same attention as its design, and we release it to the research community as an open-source, modular codebase.¹

The implementation is built on established libraries for distributed computation and Graphics Processing Unit (GPU) acceleration. Ray and its reinforcement learning library RLlib [Moritz et al., 2018, Liang et al., 2018] handle everything that involves interacting with an environment, while PyTorch

¹https://github.com/MaxAalaa/Clustering_Agent_Latent_Space

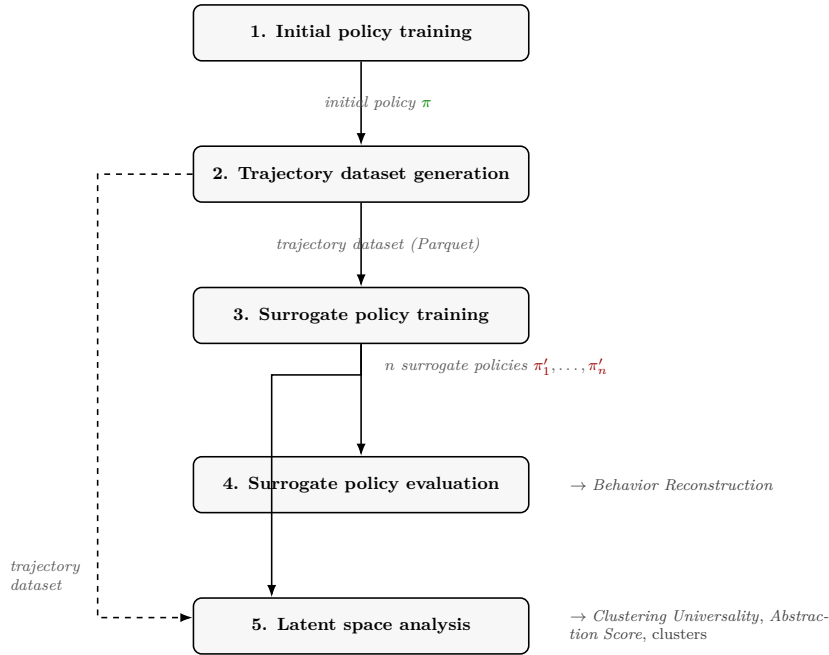


Figure 1.1: The CAR pipeline. Each stage is an independent executable; stages communicate only through artifacts written to disk (solid arrows). The dashed arrow indicates an artifact reused by a later stage.

[Paszke et al., 2019] and PyTorch Lightning [Falcon and The PyTorch Lightning team, 2019] handle the training and the analysis of the surrogate policies. Environments follow the Gymnasium interface [Towers et al., 2024], and clustering relies on scikit-learn [Pedregosa et al., 2011]. Every stage distributes its workload across parallel Central Processing Unit (CPU) workers and offloads tensor computation to the GPU whenever possible.

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate only through artifacts written to disk. This decoupling allows any stage to be rerun, inspected, or parallelized on its own, and lets a single trained policy or dataset be reused across many analyses. The parameters of a given experiment are gathered in a single configuration object, described in Section 1.2, and each stage serializes the configuration it actually used next to its outputs, so that every result can be traced back to the settings that produced it. All experiments reported in this chapter were run on the CRIANN Austral High-Performance Computing (HPC) cluster, with one batch job per experiment on NVIDIA A100 GPUs.

The environment layer wraps heterogeneous simulators behind a single interface and registers them with RLlib, so that a new environment becomes available to the whole pipeline by adding one class. More than a dozen environment families are supported, including the Atari benchmark [Bellemare et al., 2013] used for the experiments of this chapter. The remainder of this section describes each stage of the pipeline.

1.1.1 Initial policy training

The first stage trains the initial policy π with RLlib. Proximal Policy Optimization (PPO) [Schulman et al., 2017] is used by default; Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3 are also available through the same configuration. Training is parallelized: environment interactions are collected by many environment runners executing in parallel, while gradient updates are performed on the GPU by dedicated learner processes. The run is driven by Ray Tune, which periodically evaluates the policy, retains the best checkpoints according to the evaluation return, and can resume an interrupted run. The output of this stage is the checkpoint of the trained policy π .

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to imitate π . The trained policy is run in the environment and a callback records one trajectory per file. For every step, the dataset stores the observation together with the parameters of the action distribution produced by the policy, rather than only the sampled action; this is what makes it possible to imitate π with the Mean Squared Error (MSE) objective of Section ???. Trajectories are written as Parquet files, a columnar and compressed format that can be read as a stream. This is necessary because the datasets used here contain on the order of one million observation–action pairs and do not fit in memory. Dataset generation is distributed over parallel environment runners. A variant of this stage additionally stores the rendered frame of each step, later used to visualize the content of the clusters.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies by Behavioral Cloning (BC) with PyTorch Lightning. The n surrogate policies $\pi'_1, \dots, \pi'_n$ are trained independently, each from a random initialization, by minimizing the MSE between the action-distribution parameters of π and those of the surrogate. Surrogate policies use an encoder–decoder architecture, and a hook mechanism makes it possible to expose any hidden layer as the latent space to be analyzed. Training runs on the GPU with mixed-precision arithmetic, and the Parquet dataset is streamed through a data pipeline that shuffles samples by chunks and prefetches batches with several worker processes, so that the GPU is not left waiting for data. Checkpoints are saved at regular time intervals. The output is a set of n trained surrogate policies.

1.1.4 Surrogate policy evaluation

The fourth stage measures how faithfully each surrogate policy reproduces the behavior of π . Each surrogate is wrapped so that it can be used as an RLlib policy and is run back in the environment. Its mean cumulative reward over many episodes is compared to that of π , yielding the *Behavior Reconstruction* of Section ??. Evaluation episodes are collected by parallel environment runners.

1.1.5 Latent space analysis

The fifth stage performs the analysis proper. A subset of observations is projected through the encoder of each surrogate policy, at the selected layer, to produce the embedding sets $\mathcal{E}_1, \dots, \mathcal{E}_n$; this projection is computed in batches on the GPU. Each embedding set is then partitioned for a range of numbers of clusters, using k -means [MacQueen, 1967] or a Gaussian Mixture Model (GMM), with a batched implementation for large point clouds. From these partitions the stage computes the silhouette score, the *Clustering Universality* as the mean pairwise Adjusted Rand Index (ARI) between surrogate policies, and the *Abstraction Score* from the ARI between the latent clusterings and reference clusterings of the observation and action spaces. It also produces a two-dimensional projection of each latent space and, for every cluster, a set of representative observations. The outputs of this stage are the metric values and the material used for the qualitative analysis presented in the following sections.

1.2 Experimental Configuration

1.3 Results

This section quantitatively evaluates CAR across several Atari benchmark environments [Bellemare et al., 2013], which are widely used in Reinforcement Learning (RL) research [Mnih et al., 2013, Schulman et al., 2017, Zahavy et al., 2017]. The environments considered in this study are:

- **Breakout:** the agent controls a paddle to bounce a ball and break as many bricks as possible, the ball speeding up after each bounce.
- **Mario Bros:** the agent must defeat enemies, avoid fireballs, and collect mushrooms [AtariAge, 2024].
- **Pong:** a two-player game in which the agent tries to prevent the ball from passing its side while attempting to score against the opponent.
- **Space Invaders:** the agent controls a cannon and must destroy waves of invaders before they reach the ground.
- **Surround:** a two-player grid-based game in which each move blocks the traversed cell, the goal being to trap the opponent.

In all these environments, the agent receives visual observations and operates in discrete action spaces. The initial policies π are trained using PPO [Schulman et al., 2017] with a deep actor-critic architecture. The network consists of four convolutional layers with configurations (16, 4, 2), (32, 4, 2), (64, 4, 2), (128, 4, 2), where each tuple represents (number of channels, kernel size, stride), followed by three fully connected layers with 512, 256, and 128 units, respectively. To ensure that these initial policies effectively learn to maximize rewards, we follow the recommendations of [Machado et al., 2017] and compare the performance of our initial policies against the benchmark scores reported in the same study to verify that they meet the required performance standards.

Environment	<i>Behavior Reconstruction</i>	<i>Abstraction Score</i>	<i>Clustering Universality</i>
Breakout	0.83 ± 0.02	0.99 ± 0.00	0.58 ± 0.09
Mario Bros	0.98 ± 0.02	0.99 ± 0.00	0.55 ± 0.10
Pong	0.95 ± 0.00	0.99 ± 0.00	0.48 ± 0.04
Space Invaders	0.97 ± 0.07	0.99 ± 0.00	0.51 ± 0.08
Surround	0.99 ± 0.00	0.99 ± 0.00	0.58 ± 0.07

Table 1.1: Evaluation of *Behavior Reconstruction*, *Abstraction Score* and *Clustering Universality* across the five Atari environments.

For each initial policy, three surrogate policies π' sharing the same architecture are trained as described in Section ?? . These surrogate policies are trained on approximately one million observation-action pairs each.

We first evaluate the ability of the surrogate policies to replicate the behavior of the initial policies (*Behavior Reconstruction*, Table 1.1). Across all environments, the surrogate policies achieve an average *Behavior Reconstruction* of 0.94 ± 0.02 , indicating that they reliably reproduce the decisions of the initial policies over 100 evaluation episodes.

For the remainder of the analysis, a random subset of 50,000 observations is drawn from the training dataset and embedded into the Latent Space (LS) of each surrogate policy, using the representation extracted directly after the last convolutional layer. This projection results in a point cloud $\mathcal{E}$. For clustering, we opted for the simplest approach and applied **K-means clustering** [MacQueen, 1967] to the point cloud, using the Euclidean distance between the projected points to form the clusters.

Across all environments, policies, and numbers of clusters, the average *Abstraction Score* is 0.99 ± 0.00 . This indicates that the clusters formed in the LS are not mere reflections of the raw pixel inputs or of the action distributions.

Figure 1.2 reports the evolution of *Clustering Universality* with respect to the number of clusters. From three clusters onward, all curves stabilize around 0.5 in every environment. Since *Clustering Universality* is computed from the ARI, this result indicates a strong similarity in the clustering of the LS across surrogate policies within each environment [Amezquita et al., 2020], regardless of the number of clusters chosen. This stability indicates that there is no single optimal number of clusters for explaining an agent’s policy. This raises the question of how many clusters should be selected to perform an analysis of LS. Our analysis suggests that this depends on the desired level of granularity.

With fewer clusters, CAR highlights broad strategic patterns. For instance, in the Mario Bros environment, the two main groups correspond to whether the POW block is present (highlighted in red in Figure 1.3A) or not. This is particularly interesting, as the use of the POW block is a key determinant of strategy in this environment [AtariAge, 2024].

With a larger number of clusters, the analysis captures more fine-grained tactical behaviors, such as ‘Attacking the left.’ and ‘Navigating through the level.’ (see Figure 1.3B).

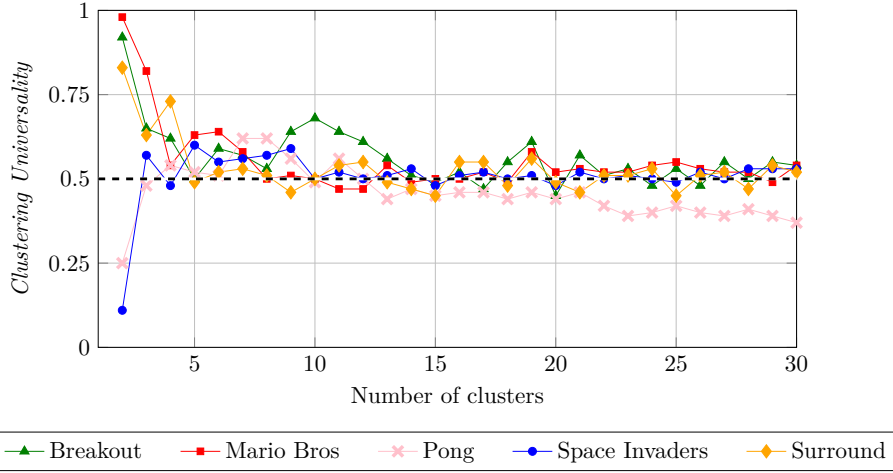


Figure 1.2: Evolution of *Clustering Universality* with respect to the number of clusters, for each of the five Atari environments.

Both perspectives are valid, depending on whether the goal is to study strategic or tactical behaviors. Similar interpretable structures can be observed across all evaluated environments.

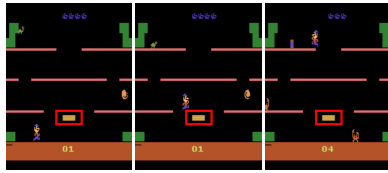
1.4 Conclusion

The experiments reported in this chapter apply CAR and its metrics to five Atari environments. The surrogate policies reproduce the behavior of the initial policies with an average *Behavior Reconstruction* of 0.94 ± 0.02 . The latent clusterings are stable across independently trained surrogate policies, with a *Clustering Universality* of about 0.5 from three clusters onward, and they are largely independent of the observation and action spaces, with an average *Abstraction Score* of 0.99 ± 0.00 . Together, these results provide empirical support for the *Representation Convergence Hypothesis* (Hyp. ??) and, more broadly, for the hypothesis-driven approach underlying CAR.

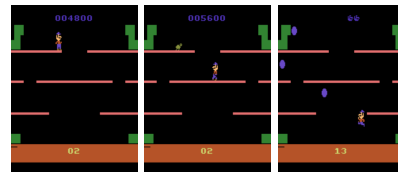
Several methodological choices have not yet been evaluated. These include the Neural Network (NN) architecture, the layer selected for analysis, the distance used to compare embeddings, and the clustering algorithm. Their impact should be studied jointly to determine how much the discovered Concept Structure (CS) depends on the analysis procedure rather than on the policy itself.

The evaluation is also limited in scope. It considers only five Atari games, and the semantic labels attached to the clusters in Figure 1.3 were assigned by hand, which is time-consuming and subjective. Extending the study to more diverse environments and developing human-centered evaluation protocols would strengthen the validity of the resulting explanations.

(A) The case with 2 clusters.



The POW block is present.

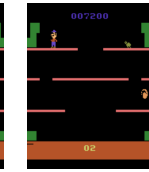
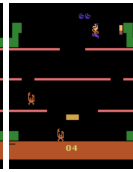


The POW block is absent.

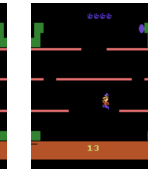
(B) The case with 5 clusters.



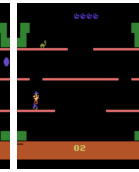
The POW block is present.



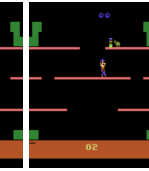
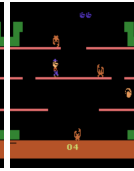
Move on the left platform.



Navigating through the level.



Attacking the left.



Attacking the right.

Figure 1.3: Effect of the number of clusters on the groupings identified by CAR in the Mario Bros environment.

195 Acronyms

196 **APPO** Asynchronous Proximal Policy Optimization 3

197 **ARI** Adjusted Rand Index 4, 5

198 **BC** Behavioral Cloning 3

199 **CAR** Clustering Agent Representation 1, 2, 4–7

200 **CPU** Central Processing Unit 2

201 **CS** Concept Structure 6

202 **DQN** Deep Q-Networks 3

203 **GMM** Gaussian Mixture Model 4

204 **GPU** Graphics Processing Unit 1–4

205 **HPC** High-Performance Computing 2

206 **LS** Latent Space 5

207 **MSE** Mean Squared Error 3

208 **NN** Neural Network 6

209 **PPO** Proximal Policy Optimization 3, 4

210 **RL** Reinforcement Learning 4

Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [AtariAge, 2024] AtariAge (2024). Mario bros. atari 2600. **Web**.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon and The PyTorch Lightning team, 2019] Falcon, W. and The PyTorch Lightning team (2019). PyTorch Lightning. <https://github.com/Lightning-AI/lightning>.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLLib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018).

- 245 Ray: A distributed framework for emerging AI applications. In 13th USENIX
246 Symposium on Operating Systems Design and Implementation (OSDI).
- 247 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
248 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
249 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
250 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
251 imperative style, high-performance deep learning library. In Advances in
252 Neural Information Processing Systems (NeurIPS).
- 253 [Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel,
254 V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg,
255 V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and
256 Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of
257 Machine Learning Research, 12:2825–2830.
- 258 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
259 Klimov, O. (2017). Proximal policy optimization algorithms.
- 260 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
261 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
262 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
263 environments.
- 264 [Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying
265 the black box: Understanding dqns. **Article**.